Over-squashing as Transport Congestion via an Abelian Sandpile Approach

Yang Shi*
sudo.shiyang@gmail.com
Guangdong University of Technology
Guangzhou, China

Lixian Chen*
lix.chen41@gmail.com
Guangdong University of Technology
Guangzhou, China

Jingchao Wang*
ethanwangjc@163.com
Peking University
Beijing, China

Minzhe Guo
capynt@gmail.com
Guangdong University of Technology
Guangzhou, China

Mingxuan Huang
huangmx53@mail2.sysu.edu.cn
Sun Yat-Sen University
Guangzhou, China

Yanhui Chen
chenyanhui91@mails.gdut.edu.cn
Guangdong University of Technology
Guangzhou, China

Yifeng Xie
evfxie@gmail.com
Hong Kong Baptist University
Hong Kong, China

Xuhang Chen
xuhangc@hzu.edu.cn
Glorious Sun Group
Huizhou, China

Liangsi Lu†
lu.liangsi.cn@gmail.com
Guangdong University of Technology
Guangzhou, China

Abstract

Message-passing graph neural networks (MP-GNNs) are widely used for learning on relational data. However, their performance drops on tasks requiring long-range interactions due to over-squashing, where exponential information compression overwhelms fixed-width embeddings. While existing analyses often attribute this to geometric bottlenecks under linear diffusion assumptions, thresholded nonlinearities in GNNs motivate a load-release view akin to Abelian sandpiles. Using the discrete sandpile model as a structural proxy, we show that graph bottlenecks force large stabilization cost, effectively creating zones of high transport congestion. We characterize stabilization-invariant equivalence classes induced by the reduced Laplacian and derive cut-based lower bounds linking bottlenecks to unavoidable stabilization effort. The resulting theory is discrete, whereas our implementation is a continuous vector-valued surrogate. The theory identifies the relevant design factors, namely capacity and cut size. Guided by these insights, we propose a differentiable **S**andpile **S**tabilization **L**ayer (**SSL**) and congestion-aware objectives designed to redistribute excess load and manage stabilization costs. Experiments on long-range benchmarks, together with congestion and collision diagnostics, show that targeting sandpile-identified bottlenecks mitigates representation collapse and improves over standard baselines.

CCS Concepts

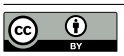
• **Computing methodologies** → *Knowledge representation and reasoning*.

Keywords

graph neural networks, over-squashing, chip-firing, Abelian sandpile, critical group, long-range dependencies

ACM Reference Format:
Yang Shi, Lixian Chen, Jingchao Wang, Minzhe Guo, Mingxuan Huang, Yanhui Chen, Yifeng Xie, Xuhang Chen, and Liangsi Lu. 2026. Over-squashing as Transport Congestion via an Abelian Sandpile Approach. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3770855.3817690

*These authors contributed equally to this research.
†Liangsi Lu is the corresponding author.



KDD '26, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817690

1 Introduction

Message-passing graph neural networks (MP-GNNs) serve as a widely used framework for representation learning on relational data [23, 40]. By iteratively aggregating neighborhood information, MP-GNNs achieve strong empirical performance across domains ranging from molecules and knowledge graphs to physical simulations and combinatorial reasoning [12, 23, 39]. A recurring requirement in these applications is the modeling of long-range interactions, where predictions depend on signals that originate many hops away, such as in multi-hop relational reasoning or global graph property prediction [20, 21, 41].

Despite their success, MP-GNNs struggle to capture these dependencies because of over-squashing. In this phenomenon, information from exponentially many remote sources must be compressed into a fixed-dimensional channel as it traverses sparse separators or low-expansion regions of the graph [1, 44]. Existing mitigation methods use expressive message functions, global attention, positional encodings, and graph rewiring strategies [10, 44], but they do not provide a discrete account of transport congestion under thresholded nonlinear dynamics.

Current theoretical analyses typically model over-squashing as a problem of linear transport or diffusion, reasoning through spectra, curvature, or Jacobian rank decay [11, 35, 44]. However, MP-GNN

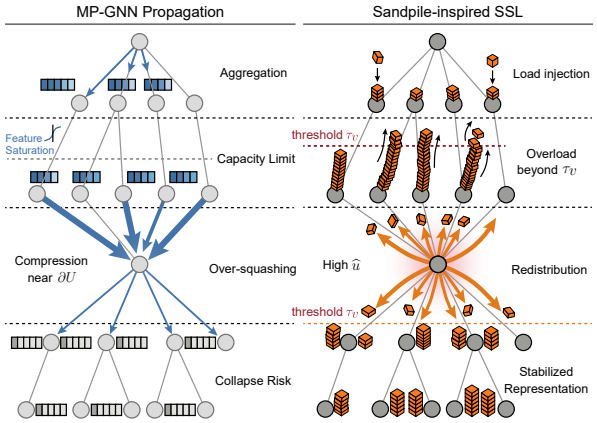


Figure 1: Conceptual comparison between MP-GNN propagation and Sandpile-inspired SSL. Left panel shows that bottlenecks in standard message passing can cause feature saturation and representation collapse risk. Right panel shows that SSL uses a learnable threshold τ_v to release overload, redistribute load, and produce an odometer-like congestion proxy $\widehat{u}$.

pipelines rely on thresholded nonlinearities that resemble a load-release mechanism rather than pure linear diffusion [1, 35]. Consequently, the field lacks a structural notion of transport congestion that yields graph-structural lower bounds under such thresholded nonlinear dynamics.

To address this gap, we propose a new theoretical lens based on chip-firing and Abelian sandpile dynamics on graphs [8, 16]. In an Abelian sandpile, vertices accumulate discrete load (chips) until they exceed a local threshold and then topple, redistributing load to neighbors. The final stabilized state and the cumulative toppling count are independent of the toppling order, a feature known as the Abelian property [16, 25]. We formalize a discrete proxy mapping from message accumulation to chip addition and from capacity-limited aggregation to sandpile stabilization. The mapping exposes capacity and cut size mechanisms rather than asserting an exact equivalence between vector-valued GNN dynamics and integer chip-firing. It exposes analyzable quantities including the odometer as the total topplings per node, which serves as a proxy for congestion, cut-crossing counts, which track flow across separators, and the Laplacian-induced critical group, which organizes stabilization equivalence classes [8, 15, 31].

Guided by this theory, we propose a differentiable **S**andpile **S**tabilization **L**ayer (**SSL**) that augments standard MP-GNN backbones with a soft stabilization step, as well as congestion-aware objectives based on odometer proxies and Sandpile-guided rewiring heuristics. Experimentally, we evaluate these methods on standard long-range benchmarks, together with congestion and collision diagnostics, reporting improvements for several MP-GNN backbones while reducing measured congestion [20].

Our main contributions are as follows.

- We define a sink-based Abelian sandpile model on graphs and analyze the termination and order-independence of its stabilization.
- We prove that if a region initially contains more load than can be stably stored, then at least that excess must cross the boundary, forcing a large odometer value on boundary nodes when the cut is small.
- We separate the sandpile-group invariants used for stabilization diagnostics from the odometer and cut-based quantities used to design SSL, congestion objectives, and rewiring heuristics.
- We empirically validate the proposed framework on long-range benchmarks, confirming a strong correlation between the sandpile congestion proxy and representation collapse, while improving performance over existing methods.

2 Related Work

2.1 Over-squashing and Long-Range Benchmarks

Over-squashing has been extensively studied as a primary structural bottleneck preventing MP-GNNs from learning long-range dependencies, with analyses typically grounded in geometric curvature, spectral gaps, or information flow limitations [1, 44]. To address this, a variety of mitigation strategies have been proposed, including graph rewiring to alter connectivity, adding edges based on expansion properties, incorporating positional encodings, and using hybrid architectures that combine local aggregation with global attention tokens [10, 24, 37, 44, 45, 47]. Additionally, dedicated long-range benchmark suites have been developed to standardize the evaluation of these methods on tasks requiring distant signal propagation [20, 34]. Unlike these approaches that primarily address topological bottlenecks through geometric or spectral proxies, our work identifies a congestion lower bound arising directly from thresholded load-release dynamics in neural message passing, providing a mechanistic explanation for signal compression failure [38].

2.2 Nonlinear Transport Views of Message Passing

Several theoretical frameworks interpret message passing neural networks as discretizations of continuous diffusion processes or partial differential equation (PDE) operators on graphs, often assuming linear or quasi-linear propagation dynamics [2, 27]. These models provide valuable insights into smoothing and stability but generally treat information flow as a continuous gradient-driven process [11, 35]. Our focus differs by modeling transport as a thresholded mechanism involving accumulation and release, motivated by the saturating nonlinearities, clipping, and normalization layers found in MP-GNN pipelines. While diffusion-based models assume continuous flows, our chip-firing framework explicitly models the capacity-limited accumulation and discrete release behaviors of GNNs, capturing hard saturation effects and transport congestion that purely linear diffusion theories overlook [42].

2.3 Foundations of Chip-Firing and Algebraic Graph Theory

The Abelian sandpile model and chip-firing dynamics are classical subjects in probability theory, combinatorics, and algebraic graph

theory, known for their self-organized criticality and algebraic properties [3, 16]. These systems exhibit rigorous structural invariants, such as the order independence of stabilization (the Abelian property) and the formation of a finite Abelian group, the critical group, determined by the reduced Laplacian [4, 8, 14, 31]. We use these established mathematical tools as an analytical foundation for graph representation learning. In particular, the Laplacian-quotient invariants characterize stabilization equivalence classes, while the odometer and cut-crossing quantities provide the congestion signals that guide SSL and rewiring. Thus the critical group is used to clarify what stabilization preserves or collapses, whereas the algorithmic components are driven by capacity, odometer, and cut size quantities.

3 Preliminaries

This section collects notation and classical tools on graphs and Abelian sandpiles that will be used throughout the paper. We intentionally separate these preliminaries from the proposed analysis and model components in Section 4.

3.1 Graph Setup and Basic Notation

Let $G = (V, E)$ be a finite, connected, undirected graph, where V is the vertex set and E is the edge set. We fix a distinguished sink vertex $s \in V$ that absorbs outgoing load.

For two sets A and B, we write $A \setminus B$ (set difference) for the set of elements in A that are not in B. We define the non-sink set

$$V^\circ \;:=\; V \setminus \{s\}. \tag{1}$$

For any set $U \subseteq V^\circ$, its complement in V is denoted by

$$U^c \;:=\; V \setminus U. \tag{2}$$

For any finite set S, $|S|$ denotes its cardinality. For $v \in V$, let $\mathcal{N}(v) := \{u \in V : \{u, v\} \in E\}$ be the neighbor set of v, and let $\deg(v) := |\mathcal{N}(v)|$ be its degree (including edges incident to the sink).

For $U \subseteq V^\circ$, the (edge) boundary of U is

$$\partial U \;:=\; \big\{\{v, w\} \in E :\; v \in U,\; w \in V \setminus U\big\}. \tag{3}$$

For $v \in U$, the external degree of v with respect to U is

$$\deg^U_{\text{out}}(v) \;:=\; \big|\{w \in V \setminus U : \{v, w\} \in E\}\big|. \tag{4}$$

Counting boundary incidences gives $\sum_{v\in U} \deg^U_{\text{out}}(v) = |\partial U|$.

We write $\mathbb{R}$ for the reals and $\mathbb{Z}$ for the integers. We define $\mathbb{Z}_{\geq 0} := \{0, 1, 2, \dots\}$. For a finite index set S, we use $\mathbb{R}^S$ to denote real-valued vectors indexed by S, and similarly $\mathbb{Z}^S$ for integer-valued vectors. Unless stated otherwise, vectors and matrices below are indexed by V°.

For $v \in V^\circ$, let $\mathbf{e}_v \in \mathbb{R}^{V^\circ}$ be the standard basis vector, and let $\mathbf{1} \in \mathbb{R}^{V^\circ}$ be the all-ones vector. Define the restricted adjacency matrix $\mathbf{A} \in \{0, 1\}^{V^\circ \times V^\circ}$ by $\mathbf{A}_{uv} = 1$ if $\{u, v\} \in E$ and $\mathbf{A}_{uv} = 0$ otherwise. Define the diagonal degree matrix $\mathbf{D} \in \mathbb{R}^{V^\circ \times V^\circ}$ by $\mathbf{D}_{vv} = \deg(v)$. The reduced (Dirichlet) Laplacian [14, 19, 32] is

$$\mathbf{L} \;:=\; \mathbf{D} - \mathbf{A} \;\in\; \mathbb{R}^{V^\circ \times V^\circ}. \tag{5}$$

For $x \in \mathbb{R}^{V^\circ}$ and $U \subseteq V^\circ$, we define the total mass of x on U as

$$x(U) \;:=\; \sum_{v\in U} x(v). \tag{6}$$

For real t, the positive-part operator is

$$[t]_+ \;:=\; \max\{t, 0\}. \tag{7}$$

For vectors $a, b \in \mathbb{R}^{V^\circ}$, $a \leq b$ means $a(v) \leq b(v)$ for every $v \in V^\circ$ (component-wise).

3.2 Abelian Sandpile with a Sink

We use the sink-based Abelian sandpile model [16, 25].

Definition 1 (Configuration and stability)**.** A configuration is a vector $z \in \mathbb{Z}^{V^\circ}_{\geq 0}$, where $z(v)$ is the number of chips (load units) at vertex $v \in V^\circ$. A vertex $v \in V^\circ$ is unstable if $z(v) \geq \deg(v)$ and stable otherwise. A configuration is stable if every vertex in V° is stable.

Definition 2 (Toppling operator)**.** For $v \in V^\circ$, define the toppling operator $T_v : \mathbb{Z}^{V^\circ} \to \mathbb{Z}^{V^\circ}$ by

$$T_v(z) \;:=\; z \;-\; \deg(v)\, \mathbf{e}_v \;+\; \sum_{u\in \mathcal{N}(v)\cap V^\circ} \mathbf{e}_u. \tag{8}$$

A toppling at v is legal if v is unstable in the current configuration. Chips sent to the sink s (along edges $\{v, s\}$) are absorbed and removed. Using (5), $T_v(z) = z - \mathbf{L}\mathbf{e}_v$.

Definition 3 (Stabilization and odometer)**.** Starting from $z \in \mathbb{Z}^{V^\circ}_{\geq 0}$, repeatedly apply legal topplings until a stable configuration is reached. The resulting stable configuration is denoted by $\text{stab}(z)$. The odometer $u_z \in \mathbb{Z}^{V^\circ}_{\geq 0}$ records how many times each vertex in V° topples during stabilization.

Theorem 1 (Termination and Abelian property)**.** *For every $z \in \mathbb{Z}^{V^\circ}_{\geq 0}$, every legal toppling sequence terminates after finitely many steps. Moreover, the stabilized configuration* $\text{stab}(z)$ *and the odometer u_z are independent of the legal toppling order.*

Theorem 2 (Least action principle)**.** *Fix $z \in \mathbb{Z}^{V^\circ}_{\geq 0}$ and let u_z be the odometer from Definition 3. For any $q \in \mathbb{Z}^{V^\circ}_{\geq 0}$ such that $z - \mathbf{L}q \in \mathbb{Z}^{V^\circ}_{\geq 0}$ is stable, we have $q \geq u_z$ component-wise. Equivalently, u_z is the component-wise minimal nonnegative vector whose application yields a stable configuration.*

We will repeatedly use the least action principle as a variational characterization. Stabilization is the component-wise minimal redistribution compatible with reaching a stable (non-overloaded) state.

Corollary 3 (Stabilization minimizes congestion among stable redistributions). *Let u_z be the odometer of z. For any $q \in \mathbb{Z}^{V^\circ}_{\geq 0}$ such that $z - \mathbf{L}q$ is stable, $q \geq u_z$ component-wise. Consequently, if we define $\|q\|_1 := \sum_{v\in V^\circ} q(v)$ and $\|q\|_\infty := \max_{v\in V^\circ} q(v)$, then*

$$\|q\|_1 \;\geq\; \|u_z\|_1, \tag{9}$$

$$\|q\|_\infty \;\geq\; \|u_z\|_\infty. \tag{10}$$

3.3 Green Function Representation

We also provide an explicit representation of stabilization effort using a killed random walk [28].

Define the random-walk kernel on V° by

$$\mathbf{P} \;:=\; \mathbf{D}^{-1}\mathbf{A}. \tag{11}$$

For $v, u \in V^\circ$, $\mathbf{P}_{vu} = 1/\deg(v)$ if $\{v, u\} \in E$ and $\mathbf{P}_{vu} = 0$ otherwise. Moves to the sink are treated as killing/absorption.

Theorem 4 (Green function and Neumann series)**.** *The reduced Laplacian* $\mathbf{L}$ *is invertible over* $\mathbb{R}$ *and*

$$\mathbf{L}^{-1} = \sum_{k=0}^{\infty} \mathbf{P}^k \mathbf{D}^{-1}. \tag{12}$$

Let $z^\circ := \mathrm{stab}(z)$*. Then*

$$u_z = \mathbf{L}^{-1}(z - z^\circ). \tag{13}$$

Moreover, define the vector $(\deg -1) \in \mathbb{R}^{V^\circ}$ *by* $(\deg -1)(v) := \deg(v) - 1$*. Then*

$$\mathbf{L}^{-1}\big(z - (\deg -1)\big) \le u_z \le \mathbf{L}^{-1} z \tag{14}$$

component-wise.

3.4 Laplacian-Quotient Invariants and Stabilization Classes

Chip-firing also induces algebraic invariants that formalize when different injections become equivalent under stabilization [8, 31].

Definition 4 (Sandpile (critical) group)**.** Define the sandpile group associated with (G, s) as

$$K(G, s) := \mathbb{Z}^{V^\circ} / \mathbf{L}\mathbb{Z}^{V^\circ}. \tag{15}$$

This is a finite Abelian group of size $|K(G, s)| = \det(\mathbf{L})$, where $\det(\cdot)$ denotes the matrix determinant. By the matrix-tree theorem, $\det(\mathbf{L})$ equals the number of spanning trees of G rooted at s [8].

For $x \in \mathbb{Z}^{V^\circ}$, we write $[x]$ for its equivalence class in $K(G, s)$. Two vectors $x, x' \in \mathbb{Z}^{V^\circ}$ are equivalent if $x - x' \in \mathbf{L}\mathbb{Z}^{V^\circ}$. Since each toppling subtracts $\mathbf{L}e_v$, stabilization preserves the class.

Proposition 1 (Stabilization preserves the Laplacian-quotient class)**.** *For any* $z \in \mathbb{Z}^{V^\circ}_{\ge 0}$*, let* $z^\circ = \mathrm{stab}(z)$*. Then* $z - z^\circ \in \mathbf{L}\mathbb{Z}^{V^\circ}$ *and hence* $[z] = [z^\circ]$ *in* $K(G, s)$*. More generally, for any* $x \in \mathbb{Z}^{V^\circ}$ *such that* $z + x \in \mathbb{Z}^{V^\circ}_{\ge 0}$*,*

$$[\mathrm{stab}(z + x)] = [z + x] \quad \text{in } K(G, s). \tag{16}$$

Role in this paper. The quotient $K(G, s)$ is used as a theoretical diagnostic, not as an algorithmic component of SSL or rewiring. Proposition 1 records that stabilization preserves the Laplacian-quotient class, while Proposition 2 below records that stabilization can collapse distinct pre-stabilization configurations to the same stable state. Together, these facts justify using sandpile stabilization to reason about post-stabilization indistinguishability, but they are not assumed by the continuous vector-valued SSL implementation.

Proposition 2 (Stabilization is many-to-one)**.** *Let* $\mathcal{S}$ *denote the set of stable configurations on* V°*, namely*

$$\mathcal{S} := \{y \in \mathbb{Z}^{V^\circ}_{\ge 0} : y(v) \le \deg(v) - 1 \text{ for all } v \in V^\circ\}. \tag{17}$$

Then $\mathcal{S}$ *is finite, while* $\mathbb{Z}^{V^\circ}_{\ge 0}$ *is infinite. Therefore the stabilization map* $\mathrm{stab} : \mathbb{Z}^{V^\circ}_{\ge 0} \to \mathcal{S}$ *is not injective because there exist* $z \ne z'$ *such that*

$$\mathrm{stab}(z) = \mathrm{stab}(z'). \tag{18}$$

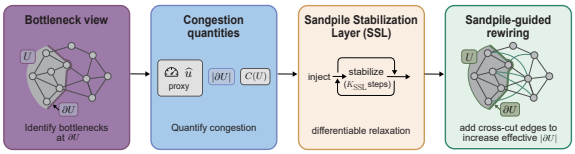


Figure 2: Overview of the Sandpile-inspired framework. The method identifies bottlenecks at graph cuts ∂U, quantifies congestion using the odometer proxy $\widehat{u}$ together with $|\partial U|$ and $C(U)$, applies SSL as a differentiable stabilization layer, and uses Sandpile-guided rewiring to increase the effective cut size $|\partial U|$.

4 Method

We now connect the sandpile toolkit from Section 3 to message passing in GNNs, derive congestion-based explanations of over-squashing, and introduce Sandpile-inspired components that directly target the resulting failure modes.

Method intuition. The proposed intervention has two connected parts. First, SSL treats each node as having a finite feature capacity. When the feature magnitude exceeds the learned threshold, only the excess component is released and locally redistributed, producing a differentiable odometer-like congestion signal. Second, rewiring uses this congestion signal to add a small number of shortcut edges from congested nodes to distant nodes, thereby enlarging the effective cut through which long-range information must pass. In short, SSL addresses the capacity factor and rewiring addresses the cut size factor suggested by the discrete theory.

Mapping between capacity-limited message passing and the sandpile proxy. We view the pre-activation/state x_v produced by message accumulation as a (continuous) load at node v. A node is unstable when $\|x_v\|_2 > \tau_v$ (capacity τ_v), quantified by overload $o_v = [\|x_v\|_2 - \tau_v]_+$. A topple corresponds to releasing only this overload r_v and redistributing it locally along edges; any discarded fraction due to clipping/saturation is modeled as mass sent to the sink, controlled by η. The cumulative intended released overload $\widehat{u}(v) = \sum_k e_v^{(k)} o_v^{(k)}$ serves as a differentiable odometer-like congestion signal.

4.1 A Sandpile View of Over-squashing

Over-squashing is the failure of message passing graph neural networks (MP-GNNs) on tasks requiring long-range interactions, where many distant sources must influence a target through a small set of edges or vertices [1, 44]. Most existing explanations emphasize graph bottlenecks such as low expansion and negative curvature that force information to funnel through narrow separators [10, 44].

We develop a complementary view in which **over-squashing is congestion under local capacity constraints**. We use the Abelian sandpile as a discrete proxy for overload release mechanisms. Load is injected, and any local overload beyond capacity is released to neighbors under a threshold rule [16, 17]. In this view, the failure mode is not merely that sources are far, but that excess load must repeatedly cross a small cut, forcing large stabilization effort concentrated near bottlenecks. This yields structural lower

bounds on unavoidable congestion and a principled way to explain why many local modifications fail unless they change the relevant cut capacity.

Figure 2 provides an overview of the pipeline.

Sink as dissipation. The sink is not an extra GNN node. It is an accounting device for irreversible loss under capacity-limited processing. Clipping discards overflow directly, saturating nonlinearities map a wide range of large inputs to nearly indistinguishable outputs, and competitive normalization can make weaker signals vanish relative to dominant ones. In SSL this effect is represented by η. Only a $(1-\eta)$ fraction of released overflow is redistributed on the data graph, while the remaining fraction is treated as dissipated mass. Thus dissipation is used for stable congestion accounting, not as a claim that downstream accuracy is governed by mass conservation alone.

4.2 Jacobian Rank Bounds Imposed by Vertex Separators

We formalize the classical bottleneck intuition using a Jacobian rank bound. If an MP-GNN remains strictly local on the original graph, then cross separator influence is rank-limited by the separator size.

Consider an L-layer MP-GNN with hidden dimension d. For any $U, W \subseteq V$, let $F_W(X) \in \mathbb{R}^{d|W|}$ denote the concatenation of final-layer embeddings on W as a differentiable function of the input X, and let X_U be the concatenation of input features on U. Define the Jacobian block

$$J_{W,U} \;:=\; \frac{\partial F_W}{\partial X_U}. \tag{19}$$

Proposition 3 (Separator Jacobian rank bound)**.** *Assume strict locality, so that each update $h_v^{(\ell+1)}$ depends only on $\{h_u^{(\ell)} : u \in \mathcal{N}(v) \cup \{v\}\}$ and on parameters. Let $\mathcal{B} \subseteq V$ be a vertex separator such that every undirected path from $U \setminus \mathcal{B}$ to $W \setminus \mathcal{B}$ intersects $\mathcal{B}$. Then*

$$\operatorname{rank}(J_{W,U}) \;\le\; d\,|\mathcal{B}|\,(L+1). \tag{20}$$

Interpretation. If a target function requires influence rank across a small separator that exceeds (20), then any architecture whose forward dependencies remain confined to local message passing on the original edge set cannot represent it, regardless of training [18].

Corollary 5 (Local modifications cannot remove a small separator bottleneck). *Any architecture whose forward dependencies remain confined to local message passing on the original edge set E (including variants with different aggregators, nonlinearities, normalization, residual connections, or local attention) still satisfies the rank bound* (20)*. Therefore, for tasks whose target function requires cross separator dependence rank exceeding the right-hand side of* (20)*, such methods cannot represent the function regardless of training.*

The proof is immediate from Proposition 3 and is omitted.

4.3 Cuts Force Congestion

We now connect bottlenecks to forced stabilization effort.

Definition 5 (Region stable capacity)**.** For $U \subseteq V^\circ$, define the stable capacity

$$C(U) \;:=\; \sum_{v\in U} \big(\deg(v) - 1\big). \tag{21}$$

This equals the maximum number of chips that can be stored in U by any stable configuration, since stability enforces $z(v) \le \deg(v) - 1$ for each $v \in U$.

We refer to $C(U)$ as the structural (degree-implied) storage capacity of U under the discrete sandpile stability constraint $z(v) \le \deg(v) - 1$. In SSL (Section 4.6), capacities are represented by learnable thresholds $\{\tau_v\}$ acting on feature norms; this constitutes a representation capacity that can be tuned (or tied to degree, for example $\tau_v = \kappa(\deg(v) - 1)$) to mirror the discrete limit while remaining differentiable and vector-valued.

For $U \subseteq V^\circ$ and $z \in \mathbb{Z}_{\ge 0}^{V^\circ}$, define the total number of chips injected into U as $z(U) = \sum_{v\in U} z(v)$ (consistent with (6)). If $z(U) > C(U)$, then any stable outcome must send at least $z(U) - C(U)$ chips out of U through ∂U. Since each toppling at $v \in U$ can send at most $\deg_{\text{out}}^U(v)$ chips across the boundary, we obtain a cut-forced lower bound on the total number of topplings in U.

Boundary crossings. We denote by $N_{U\to U^c}(z)$ the total number of chips that cross the boundary from U to U^c during stabilization; equivalently,

$$N_{U\to U^c}(z) \;:=\; \sum_{v\in U} u_z(v)\;\deg_{\text{out}}^U(v). \tag{22}$$

Theorem 6 (Cut-forced lower bound on congestion)**.** *Fix a nonempty set $U \subseteq V^\circ$ and $z \in \mathbb{Z}_{\ge 0}^{V^\circ}$, and let u_z be the odometer. Then*

$$\sum_{v\in U} u_z(v)\;\deg_{\text{out}}^U(v) \;\ge\; \big[z(U) - C(U)\big]_+. \tag{23}$$

In particular, since G is connected and $s \notin U$, nonempty U satisfies $|\partial U| > 0$; using $\sum_{v\in U} \deg_{\text{out}}^U(v) = |\partial U|$, we have

$$\|u_z\|_\infty \;\ge\; \frac{\big[z(U) - C(U)\big]_+}{|\partial U|}. \tag{24}$$

Theorem 6 shows that if many sources inject load into a region U but $|\partial U|$ is small, then some node must topple many times. This is the sandpile analogue of transport congestion through a small cut, and it is unavoidable.

Implication for the method. The displayed lower bound separates the two structural factors used by the method. For a fixed injected load $z(U)$, the numerator $[z(U) - C(U)]_+$ can be reduced only by increasing effective storage capacity in U or by reducing the load that must be routed through U. The denominator can be improved only by increasing the effective boundary through which load exits U. Edges added entirely inside U or entirely inside U^c may improve local mixing, but they do not directly change $|\partial U|$ for this bottleneck and therefore do not weaken the cut bound. This explains why SSL and rewiring are complementary. SSL acts on capacity and overflow, while Sandpile-guided rewiring is designed to add cross-region shortcuts that enlarge bottleneck boundaries under a fixed budget.

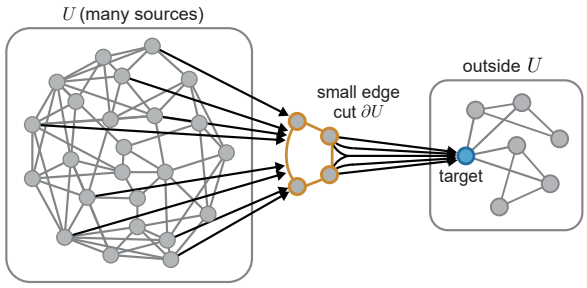


Figure 3: Illustration of a structural bottleneck induced by a small edge cut. Signals from many sources in U must pass through a small edge boundary ∂U before reaching a target outside U, creating congestion near the bottleneck.

4.4 Structural Limitations of Existing Mitigation Strategies

We now answer the question why other methods cannot solve it in the new view. The cut lower bound (24) exposes two structural factors that reduce forced congestion for a given region U and injection z. A method must increase $C(U)$ as effective storage capacity or increase $|\partial U|$ as effective cut size. Any method that does neither cannot remove the lower bound.

Depth, residual connections, and local attention do not change the cut. Increasing depth L does not change $|\partial U|$. Residual connections preserve locality and thus cannot increase boundary capacity. Local attention reweights existing edges but does not add new ones; it does not increase $|\partial U|$ either. Therefore, these modifications cannot remove the cut-forced lower bound.

Nonlinearities and normalization do not necessarily increase effective capacity. Many nonlinearities are saturating, which can reduce distinguishability and act like a capacity limit. Under saturating capacity-limited nonlinearities, repeated boundary updates can shrink sensitivity. Normalization can rescale but cannot create new channels across ∂U. Thus, while such operations can help optimization, they do not change the structural factors that govern forced congestion [5].

Positional encodings and structural features are not a substitute for transport capacity. Adding positional encodings may help identify nodes, but it does not create additional routes for information to cross a bottleneck. If the task requires aggregating many distant sources, the cut size still limits how much distinct influence can be delivered. Figure 3 illustrates this small-cut bottleneck geometry.

Global attention and fully connected mixing change the cut but are expensive. Architectures with true global mixing effectively make $|\partial U|$ large (or remove the notion of a small cut), which can avoid the lower bound. However, they incur high computational and memory costs and may be undesirable in large graphs. Our approach aims to obtain targeted increases in effective cut size with minimal overhead.

4.5 Continuous Relaxation of Discrete Sandpile Dynamics

Theorems in Sections 4.3 and 3.4 are stated for integer-valued configurations $z \in \mathbb{Z}_{\geq 0}^{V^\circ}$ under discrete chip-firing topplings, where a toppling at v sends a fixed amount (one chip per incident edge) to neighbors [17]. SSL, in contrast, operates on vector features $\{x_v \in \mathbb{R}^d\}_{v \in V^\circ}$ and redistributes only overflow beyond a learned threshold (Section 4.6), which is closer in spirit to the divisible (continuous-mass) sandpile than to the strictly integer model.

We therefore use two connected levels. The first level is discrete structural analysis. It proves unavoidable congestion statements for chip-firing dynamics such as cut-forced lower bounds on boundary crossings in Theorem 6. These results are combinatorial and depend only on (G, s), giving structural constraints. If a region U receives more load than it can stably store and has a small boundary $|\partial U|$, then some boundary node must incur large stabilization effort.

The second level is differentiable intervention. It constructs an overload release layer that follows the same qualitative mechanism. Each node keeps up to a capacity level and ships excess to neighbors. When features are scalar, nonnegative, and $\tau_v = \deg(v) - 1$ with a full release gate, SSL becomes a smooth relaxation of a parallelizable stabilization step. In the general vector-valued case, it redistributes overflow to reduce repeated saturation and exposes an internal congestion surrogate that can be regularized and used for rewiring.

The discrete and continuous layers have different roles in the argument. The discrete sandpile model identifies storage capacity, boundary size, and odometer-like congestion as the structural factors that govern bottlenecks. The implemented SSL uses the same factors through feature-norm capacity, local overflow release, and cut-aware redistribution. The resulting congestion proxy is used for regularization and rewiring, while the learned representation after K_{SSL} steps is passed to the downstream predictor. Appendix B.1 gives the detailed scope of this relaxation.

Sensitivity interpretation. This bridge also explains the representation-collision diagnostic. Suppose a capacity-limited node map g_{τ_v} satisfies $\|Dg_{\tau_v}(x)\|_{\mathrm{op}} \leq \epsilon_{\mathrm{sat}} < 1$ whenever $\|x\|_2 \geq \tau_v$, and other local maps along a path have operator norm at most L_0. If an influence path crosses m saturated boundary updates along a path of length T, then the chain rule gives

$$\left\| \frac{\partial F}{\partial X_U} \right\|_{\mathrm{op}} \leq L_0^{T-m} \epsilon_{\mathrm{sat}}^m. \tag{25}$$

Thus high cut-forced congestion should coincide with suppressed sensitivity and near-collisions, which is precisely what Figure 4 tests empirically. The diagnostic therefore connects the discrete lower bound to the observed collapse pattern, while the scalar statement and proof are kept in the appendix.

4.6 The Sandpile Stabilization Layer and Sandpile-Guided Rewiring

We now introduce Sandpile-inspired components that directly target the two factors suggested by the theory. The first component uses node capacity through stabilization-like overflow redistribution as in Corollary 3. The second component increases effective cut size by adding a small number of shortcut edges as in Proposition 4.

4.6.1 Sandpile Stabilization Layer (SSL). Let d be the hidden dimension and let K_{SSL} be the number of SSL stabilization iterations. For each non-sink node $v \in V^\circ$, let $h_v \in \mathbb{R}^d$ be its input feature. We define a learnable capacity $\tau_v \geq 0$ (scalar) and a release gate $e_v \in [0,1]$ that controls how much overload is redistributed. We interpret $\|h_v\|_2$ as load magnitude (any norm can be used; we use ℓ_2 by default).

Overload and release. At iteration k, define the overload magnitude

$$o_v^{(k)} \;:=\; \big[\|x_v^{(k)}\|_2 - \tau_v\big]_+, \tag{26}$$

and define a release fraction

$$e_v^{(k)} \;:=\; \sigma(\alpha_{\mathrm{gate}}\, o_v^{(k)} + b_{\mathrm{gate}}), \tag{27}$$

where $\sigma(\cdot)$ is the sigmoid, b_{gate} is a learnable bias, and $\alpha_{\mathrm{gate}} = \mathrm{softplus}(\widetilde{\alpha}_{\mathrm{gate}}) \geq 0$ is a learnable nonnegative slope so that the release fraction is monotone in overload. For backpropagation, we implement the hard threshold $[\cdot]_+$ with a smooth approximation.

The released vector should correspond to the excess beyond capacity (sandpile-style toppling removes overflow, not the entire mass). We therefore release only the overload component along $x_v^{(k)}$.

$$r_v^{(k)} \;:=\; e_v^{(k)} \cdot \frac{o_v^{(k)}}{\|x_v^{(k)}\|_2 + \varepsilon_{\mathrm{num}}}\, x_v^{(k)}, \tag{28}$$

where $\varepsilon_{\mathrm{num}} > 0$ is a small constant for numerical stability.

Redistribution step. Let $d_v := |\mathcal{N}(v) \cap V^\circ|$ denote the degree of v in the non-sink (data) graph. We send the released overload equally to non-sink neighbors when $d_v > 0$, with optional dissipation rate $\eta \in [0,1]$ as

$$\delta_{v\to u}^{(k)} \;:=\; \frac{(1-\eta)}{d_v}\, r_v^{(k)} \quad \text{for } u \in \mathcal{N}(v) \cap V^\circ \text{ and } d_v > 0. \tag{29}$$

If $d_v = 0$, no non-sink redistribution is performed and the released overload is treated as fully dissipated. The feature state is then updated by

$$x_v^{(k+1)} \;:=\; x_v^{(k)} - r_v^{(k)} + \sum_{u \in \mathcal{N}(v)\cap V^\circ} \delta_{u\to v}^{(k)}. \tag{30}$$

Unlike integer chip-firing, this vector-valued update does not conserve the total feature norm $\sum_v \|x_v\|_2$. Redistributed vectors may aggregate with different directions and partially cancel. Thus, the discrete cut lower bound motivates capacity and cut size control rather than serving as a literal lower bound on vector-norm overload in SSL. The quantity $\widehat{u}(v) := \sum_{k=0}^{K_{\mathrm{SSL}}-1} e_v^{(k)}\, o_v^{(k)}$ is an odometer proxy. It measures cumulative intended released overflow from v and serves as a differentiable proxy for congestion. Because of the normalization by $\|x_v^{(k)}\|_2 + \varepsilon_{\mathrm{num}}$, the actual released norm at step k is $e_v^{(k)} o_v^{(k)} \|x_v^{(k)}\|_2 / (\|x_v^{(k)}\|_2 + \varepsilon_{\mathrm{num}}) \leq e_v^{(k)} o_v^{(k)}$, so $\widehat{u}$ upper-bounds the accumulated released norm. Since $\widehat{u}$ is computed from learned feature norms rather than integer chip counts, it is a feature-dependent, topology-informed bottleneck signal. Large values indicate repeated capacity violation at a node, which may reflect both graph bottlenecks and task-dependent feature magnitude, while the post-stabilization feature $x^{(K_{\mathrm{SSL}})}$ remains the representation passed to downstream layers. Even when scalar accumulated release is monotone in K_{SSL}, the vector update contains both subtraction and redistribution, so coordinates and directions of $x^{(K_{\mathrm{SSL}})}$ need not move monotonically. This separation is important for the non-monotone K_{SSL} behavior in Figure 4. Increasing K_{SSL} advances the relaxation trajectory, but it does not monotonically increase expressive power or guarantee improved validation performance.

Stopping and output. Run the above for K_{SSL} iterations, with small values such as $K_{\mathrm{SSL}} \in \{2,3,5\}$, and output

$$\mathrm{SSL}(h)_v \;:=\; x_v^{(K_{\mathrm{SSL}})}. \tag{31}$$

We optionally concatenate $\widehat{u}(v)$ to the node embedding to expose bottleneck activity to downstream layers, and summarize congestion by $\|\widehat{u}\|_\infty := \max_{v\in V^\circ} \widehat{u}(v)$ and $\|\widehat{u}\|_1 := \sum_{v\in V^\circ} \widehat{u}(v)$.

4.6.2 Congestion regularization. We introduce a regularization term on the odometer-like signals to mitigate severe, localized congestion at graph bottlenecks. By defining the accumulated per-node congestion as $\widehat{u}(v) = \sum_{k=0}^{K_{\mathrm{SSL}}-1} e_v^{(k)}\, o_v^{(k)}$, we add

$$\mathcal{L}_{\mathrm{cong}} \;:=\; \lambda_1 \sum_{v\in V^\circ} \widehat{u}(v) \;+\; \lambda_\infty \max_{v\in V^\circ} \widehat{u}(v), \tag{32}$$

where $\lambda_1, \lambda_\infty \geq 0$ are hyperparameters. This is inspired by the discrete minimality in (10), using the surrogate $\widehat{u}$.

4.6.3 Sandpile-guided rewiring. To act on the structural factor $|\partial U|$, we add a small number of shortcut edges guided by congestion hotspots. Let $\widehat{u}(v)$ be the learned congestion signal from SSL and let B_{edge} be a per-graph edge-addition budget. We form ordered candidates (v,u), where v is selected from high-congestion source nodes, $u \in V^\circ$ is a non-neighbor of v, and the added graph edge is the undirected pair $\{v,u\}$. The candidate score is

$$\mathrm{Score}_G(v,u) \;:=\; \widehat{u}(v)\, \mathrm{dist}_G(v,u), \tag{33}$$

where dist_G is the shortest-path distance in the original graph G. Intuitively, this favors connecting highly congested nodes to far-away nodes, encouraging long shortcuts that are likely to cross bottlenecks while respecting a small edge budget. The greedy selection uses this ordered score under the fixed edge budget and skips duplicate or already connected node pairs.

Proposition 4 (Cross-cut edges increase effective boundary)**.** *Fix a nonempty region $U \subseteq V^\circ$ and add b_{cut} new simple edges, each with one endpoint in U and the other in $V^\circ \setminus U$, to obtain G'. For any graph H on the same vertex set and sink, write $\deg_H(v)$ for the degree in H, $C_H(U) := \sum_{v\in U}(\deg_H(v) - 1)$, $\partial_H U$ for the boundary in H, and u_z^H for the odometer obtained by stabilizing z on H. Then*

$$|\partial_{G'} U| \;=\; |\partial_G U| + b_{\mathrm{cut}}, \qquad C_{G'}(U) \;=\; C_G(U) + b_{\mathrm{cut}}. \tag{34}$$

Consequently, applying the cut bound (24) *on G' yields*

$$\|u_z^{G'}\|_\infty \;\geq\; \frac{\big[z(U) - C_{G'}(U)\big]_+}{|\partial_{G'} U|} \;=\; \frac{\big[z(U) - C_G(U) - b_{\mathrm{cut}}\big]_+}{|\partial_G U| + b_{\mathrm{cut}}}. \tag{35}$$

Thus cross-cut rewiring weakens the cut-forced lower bound through both structural factors. It enlarges the boundary denominator and, under degree-implied capacity, increases the stable storage capacity of U.

Table 1: Results on LRGB benchmarks [20]. All entries are mean±std over 5 seeds. The best results are in underlined. Additional matched controls are reported only for Peptides-func under the GCN setting.

Method	VOC-SP (macro-F1↑)	COCO-SP (macro-F1↑)	Peptides-func (AP↑)	Peptides-struct (MAE↓)
Message-passing baselines				
GCN	0.1268 ± 0.0031	0.0841 ± 0.0006	0.5872 ± 0.0023	0.3496 ± 0.0013
GIN	0.1265 ± 0.0042	0.1339 ± 0.0007	0.6373 ± 0.0022	0.3545 ± 0.0045
GatedGCN	0.2873 ± 0.0217	0.2529 ± 0.0015	0.6850 ± 0.0049	0.3420 ± 0.0013
Long-range baselines				
GraphGPS	0.3727 ± 0.0104	0.3485 ± 0.0032	0.6537 ± 0.0050	0.2500 ± 0.0012
Exphormer	0.3960 ± 0.0010	0.3461 ± 0.0005	0.6610 ± 0.0023	0.2492 ± 0.0010
GRIT	0.3886 ± 0.0010	0.3642 ± 0.0018	0.6978 ± 0.0082	0.2460 ± 0.0012
SAN	0.4050 ± 0.0013	0.3820 ± 0.0010	0.6900 ± 0.0045	0.2391 ± 0.0010
GRASS	0.5442 ± 0.0012	0.4674 ± 0.0020	0.6731 ± 0.0035	0.2445 ± 0.0012
FloydNet	0.4200 ± 0.0016	0.3920 ± 0.0013	0.6820 ± 0.0038	0.2357 ± 0.0009
Rewiring / mitigation baselines				
SDRF	0.1251 ± 0.0031	0.0830 ± 0.0006	0.5851 ± 0.0025	0.3404 ± 0.0013
DIGL	0.2921 ± 0.0100	0.2317 ± 0.0012	0.6830 ± 0.0030	0.2616 ± 0.0013
GOKU	0.3520 ± 0.0080	0.2760 ± 0.0020	0.6940 ± 0.0030	0.2665 ± 0.0012
PairNorm	–	–	0.6680	–
DGN	–	–	0.6710	–
Soft clip + redistribute	–	–	0.6820	–
FoSR	–	–	0.6850	–
LASER	–	–	0.6870	–
Ours				
GCN + SSL + Rewire	0.2850 ± 0.0090	0.2580 ± 0.0042	0.6994 ± 0.0023	0.2580 ± 0.0030
GIN + SSL + Rewire	0.3050 ± 0.0075	0.2780 ± 0.0033	0.7023 ± 0.0011	0.2420 ± 0.0016
GatedGCN + SSL + Rewire	0.3350 ± 0.0055	0.3080 ± 0.0022	0.7050 ± 0.0020	0.2405 ± 0.0014
GraphGPS + SSL + Rewire	0.4120 ± 0.0022	0.3840 ± 0.0018	0.7120 ± 0.0016	0.2355 ± 0.0012

5 Experiments

We evaluate whether the proposed **Sandpile Stabilization Layer** (SSL) and **Sandpile-guided rewiring** mitigate long-range failure modes (over-squashing/overload) predicted by the sandpile view, and whether this translates into improved downstream performance on long-range graph benchmarks.

5.1 Experimental Setup

Datasets and evaluation metrics. In accordance with the LRGB framework [20], we assess our model on four real-world datasets characterized by long-range dependencies. Specifically, we perform node classification on VOC-SP and COCO-SP, multi-label graph classification on Peptides-func, and graph regression on Peptides-struct. We adopt the standard LRGB evaluation metrics: macro-F1 for both VOC-SP and COCO-SP, Average Precision (AP) for Peptides-func, and Mean Absolute Error (MAE) for Peptides-struct [20]. Final test set evaluations are conducted using the model checkpoint that achieves the highest validation score on the respective primary metric. All reported results represent the mean ± standard deviation across 5 independent random seeds.

Implementation details. We integrate SSL as an independent stabilization layer positioned between the backbone and the readout, optionally employing Sandpile-guided rewiring constrained by a per-graph edge budget of $B_{\text{edge}} = \rho_{\text{edge}}|E|$. To ensure equitable comparisons, the underlying backbone architectures, LRGB data splits, and training protocols remain identical across all evaluated methods. For our approach, the hyperparameters K_{SSL} and η are held constant, whereas ρ_{edge} is tuned via validation on a restricted set of candidates. We log and aggregate congestion diagnostics—such as the norms of $\widehat{u}$—at the best-performing validation checkpoint , strictly maintaining consistent definitions and logging procedures across all runs. Comprehensive algorithmic and numerical details are provided in Appendix A. Additionally, Table 1 presents matched controls specifically for Peptides-func, and Table 3 details the distance computation controls utilized during rewiring. All computational experiments are executed on a single NVIDIA H20 GPU.

5.2 Comparison Experiment

Baselines. Our comparative evaluation features a broad spectrum of modeling strategies for long-range graph dependencies. As foundational baselines, we utilize standard MP-GNNs, specifically GCN [27], GIN [46], and GatedGCN [9]. To represent state-of-the-art long-range capabilities, we include models leveraging global or hybrid mechanisms, namely GraphGPS [36], Exphormer [43], GRIT [33], SAN [13], GRASS [30], and FloydNet [48]. Furthermore, we benchmark against topology-modifying interventions such as SDRF [44], DIGL [22], and GOKU [29]. To rigorously isolate the benefits of our proposed SSL and Sandpile-guided rewiring, we introduce targeted controls on the Peptides-func dataset under the GCN setting. By including PairNorm [49], DGN [7], a soft clipping

Table 2: Component ablation study.

Components			Test metric	$\|\widehat{u}\|_\infty$	$\|\widehat{u}\|_1$
GCN	SSL	Rewire			
✓	✗	✗	0.5872 ± 0.0023	1.00 ± 0.09	1.00 ± 0.11
✓	✓	✗	0.6852 ± 0.0038	0.39 ± 0.06	0.36 ± 0.05
✓	✗	✓	0.6192 ± 0.0048	0.76 ± 0.09	0.76 ± 0.07
✓	✓	✓	0.6994 ± 0.0023	0.27 ± 0.04	0.29 ± 0.05

baseline, FoSR [26], and LASER [6], we rule out alternative explanations for performance gains, such as normalization or generic graph rewiring. We implement our approach across both local and hybrid (GraphGPS) architectures, ensuring fair comparison through matched splits, training budgets, and topological modification budgets.

Discussion. Across the four benchmark datasets, our integrated framework reliably achieves or exceeds the performance of robust baseline models (Table 1). The benefits of our approach extend across model paradigms, delivering clear improvements for standard local MP-GNNs while still offering supplementary gains to advanced hybrid models such as GraphGPS. When evaluated in a strict GCN setting on the Peptides-func dataset, our method outshines a variety of targeted controls, including alternative rewiring techniques, soft clipping, directional messaging, and normalization. Supported by our controlled experiments and ablations, these results substantiate that our performance boosts are explicitly rooted in targeted bottleneck relief rather than a generic increase in graph density.

5.3 Ablation and Sensitivity Analysis

We conduct ablations in a representative setting, using GCN as the backbone and evaluating on the Peptides-func dataset.

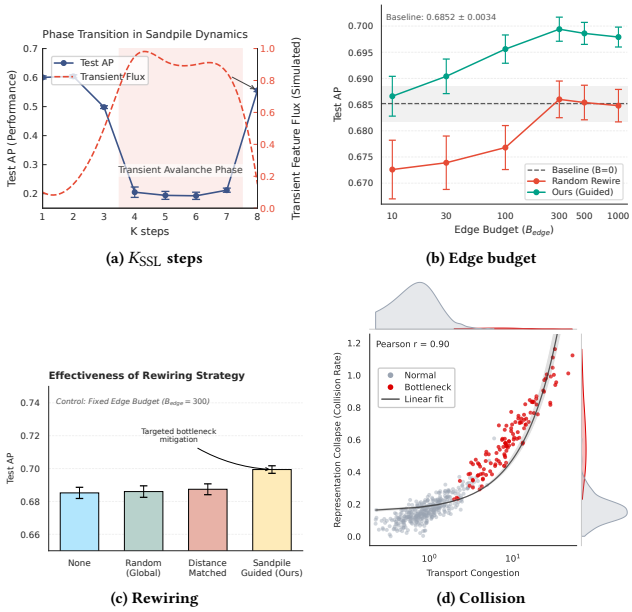
Table 2 isolates the contributions of SSL and rewiring. To decouple rewiring from the runtime effect of SSL, the Rewire-only variant rewires graphs using the odometer proxy $\widehat{u}$ computed from a Backbone+SSL run at the selected validation checkpoint, and then retrains the backbone without SSL on the rewired graphs. This makes the rewiring step task aware rather than purely unsupervised structural preprocessing; comparisons with static rewiring baselines should therefore be read as fixed-budget structural controls rather than claims that all preprocessing signals use identical supervision.

Table 3 separates the effect of distance computation from the effect of rewiring. Shortest-path distances are used only to construct the rewired graph before training. Under the same edge budget, landmark approximation reduces preprocessing time and leaves AP nearly unchanged. The per-epoch training overhead is zero in both variants because the graph is fixed after preprocessing.

The rewiring panels compare Sandpile-guided edges with random and distance-matched controls. The distance-matched control samples non-edges with the same original shortest-path length distribution as the Sandpile-guided added edges, but randomizes the endpoints. This controls for generic shortcut length and isolates whether selecting high-congestion endpoints matters. To measure

Table 3: Preprocessing comparison for rewiring distance computation.

Variant	Preprocess	Train ovhd.	Test AP (%)
Exact BFS	12.4 ms/graph	0	69.1
Landmark approximation	4.3 ms/graph	0	68.9



(a) K_{SSL} steps (b) Edge budget (c) Rewiring (d) Collision

Figure 4: Ablation and mechanism diagnostics for stabilization depth, edge budget, rewiring strategy, and congestion-collision correlation.

representation collapse, we define a near-collision diagnostic over label-discordant node pairs:

$$\text{NearColl}(\theta_{\text{coll}}) = \frac{1}{|\mathcal{P}_{\text{disc}}|} \sum_{(u,v)\in\mathcal{P}_{\text{disc}}} \mathbf{1}\left[\frac{h_u^\top h_v}{\|h_u\|\|h_v\|} > \theta_{\text{coll}}\right], \quad (36)$$

where $\mathcal{P}_{\text{disc}}$ contains label-discordant node pairs that should remain distinguishable.

Figure 4 presents our sensitivity and mechanism diagnostics. We observe that K_{SSL} acts non-monotonically, illustrating the layer's relaxation path rather than a lack of model capacity. In terms of rewiring, the Sandpile-guided approach peaks at 0.6994 AP ($B_{\text{edge}} = 300$), whereas baselines show little improvement. The NearColl diagnostic in Eq. (36) also reveals a strong correlation (Pearson $r = 0.90$) between max-congestion $\|\widehat{u}\|_\infty$ and representation collision, validating our congestion hypothesis. Finally, since K_{SSL} and η are fixed per backbone in our main results, these sweeps act strictly as mechanism diagnostics, not benchmark retuning.

6 Conclusion

In this work, we introduced Abelian sandpile dynamics to structurally analyze transport congestion in MP-GNNs. Our discrete theoretical framework reveals that node capacity and cut size directly drive stabilization costs, which we operationalize through

our trainable SSL and Sandpile-guided rewiring modules. Empirical results demonstrate that alleviating these bottlenecks enhances the performance of both local and hybrid architectures, successfully bridging the discrete theory with continuous implementations. Crucially, our benchmark evaluations and ablations confirm that these improvements stem from precise bottleneck mitigation rather than mere graph densification or auxiliary smoothing effects.

References

[1] Uri Alon and Eran Yahav. 2020. On the bottleneck of graph neural networks and its practical implications. *arXiv preprint arXiv:2006.05205* (2020).

[2] James Atwood and Don Towsley. 2016. Diffusion-convolutional neural networks. *Advances in neural information processing systems* 29 (2016).

[3] P Bak, C Tang, and K Wiesenfeld. [n. d.]. Self-organized criticality: an explanation of 1/f noise, 1987. *Phys. Rev. Lett* 59 ([n. d.]), 381.

[4] Matthew Baker and Serguei Norine. 2007. Riemann–Roch and Abel–Jacobi theory on a finite graph. *Advances in Mathematics* 215, 2 (2007), 766–788.

[5] Julia Balla. 2023. Over-Squashing in Riemannian Graph Neural Networks. *arXiv preprint arXiv:2311.15945* (2023).

[6] Federico Barbero, Ameya Velingker, Amin Saberi, Michael Bronstein, and Francesco Di Giovanni. 2023. Locality-aware graph-rewiring in gnns. *arXiv preprint arXiv:2310.01668* (2023).

[7] Dominique Beaini, Saro Passaro, Vincent Létourneau, Will Hamilton, Gabriele Corso, and Pietro Liò. 2021. Directional graph networks. In *International Conference on Machine Learning*. PMLR, 748–758.

[8] Norman L Biggs. 1999. Chip-firing and the critical group of a graph. *Journal of Algebraic Combinatorics* 9, 1 (1999), 25–45.

[9] Xavier Bresson and Thomas Laurent. 2017. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553* (2017).

[10] Rickard Brüel-Gabrielsson, Mikhail Yurochkin, and Justin Solomon. 2022. Rewiring with positional encodings for graph neural networks. *arXiv preprint arXiv:2201.12674* (2022).

[11] Chen Cai and Yusu Wang. 2020. A note on over-smoothing for graph neural networks. *arXiv preprint arXiv:2006.13318* (2020).

[12] Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. 2023. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research* 24, 130 (2023), 1–61.

[13] Dexiong Chen, Leslie O'Bray, and Karsten Borgwardt. 2022. Structure-aware transformer for graph representation learning. In *International conference on machine learning*. PMLR, 3469–3489.

[14] Fan RK Chung. 1997. *Spectral graph theory*. Vol. 92. American Mathematical Soc.

[15] Alessandra Cipriani, Rajat Subhra Hazra, and Wioletta M Ruszel. 2018. Scaling limit of the odometer in divisible sandpiles. *Probability theory and related fields* 172, 3 (2018), 829–868.

[16] Deepak Dhar. 1990. Self-organized critical state of sandpile automaton models. *Physical Review Letters* 64, 14 (1990), 1613.

[17] Deepak Dhar. 1999. The abelian sandpile and related models. *Physica A: Statistical Mechanics and its applications* 263, 1-4 (1999), 4–25.

[18] Francesco Di Giovanni, T Konstantin Rusch, Michael M Bronstein, Andreea Deac, Marc Lackenby, Siddhartha Mishra, and Petar Veličković. 2023. How does over-squashing affect the power of GNNs? *arXiv preprint arXiv:2306.03589* (2023).

[19] Peter G Doyle and J Laurie Snell. 1984. *Random walks and electric networks*. Vol. 22. American Mathematical Soc.

[20] Vijay Prakash Dwivedi, Ladislav Rampášek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. 2022. Long range graph benchmark. *Advances in Neural Information Processing Systems* 35 (2022), 22326–22340.

[21] Yanlin Feng, Xinyue Chen, Bill Yuchen Lin, Peifeng Wang, Jun Yan, and Xiang Ren. 2020. Scalable multi-hop relational reasoning for knowledge-aware question answering. *arXiv preprint arXiv:2005.00646* (2020).

[22] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. 2019. Diffusion improves graph learning. *Advances in neural information processing systems* 32 (2019).

[23] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. Pmlr, 1263–1272.

[24] Benjamin Gutteridge, Xiaowen Dong, Michael M Bronstein, and Francesco Di Giovanni. 2023. Drew: Dynamically rewired message passing with delay. In *International Conference on Machine Learning*. PMLR, 12252–12267.

[25] Alexander E Holroyd, Lionel Levine, Karola Mészáros, Yuyal Peres, James Propp, and David B Wilson. 2008. Chip-firing and rotor-routing on directed graphs. In *In and out of equilibrium 2*. Springer, 331–364.

[26] Kedar Karhadkar, Pradeep Kr Banerjee, and Guido Montúfar. 2022. FoSR: First-order spectral rewiring for addressing oversquashing in GNNs. *arXiv preprint arXiv:2210.11790* (2022).

[27] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[28] Lionel Levine and Yuval Peres. 2017. Laplacian growth, sandpiles, and scaling limits. *Bull. Amer. Math. Soc.* 54, 3 (2017), 355–382.

[29] Langzhang Liang, Fanchen Bu, Zixing Song, Zenglin Xu, Shirui Pan, and Kijung Shin. 2025. Mitigating Over-Squashing in Graph Neural Networks by Spectrum-Preserving Sparsification. *arXiv preprint arXiv:2506.16110* (2025).

[30] Tongzhou Liao and Barnabás Póczos. 2024. Greener GRASS: Enhancing GNNs with Encoding, Rewiring, and Attention. *arXiv preprint arXiv:2407.05649* (2024).

[31] Dino Lorenzini. 2008. Smith normal form and Laplacians. *Journal of Combinatorial Theory, Series B* 98, 6 (2008), 1271–1300.

[32] Russell Lyons and Yuval Peres. 2017. *Probability on trees and networks*. Vol. 42. Cambridge University Press.

[33] Liheng Ma, Chen Lin, Derek Lim, Adriana Romero-Soriano, Puneet K Dokania, Mark Coates, Philip Torr, and Ser-Nam Lim. 2023. Graph inductive biases in transformers without message passing. In *International Conference on Machine Learning*. PMLR, 23321–23337.

[34] Ivan Marisca, Jacob Bamberger, Cesare Alippi, and Michael M Bronstein. 2025. Over-squashing in Spatiotemporal Graph Neural Networks. *arXiv preprint arXiv:2506.15507* (2025).

[35] Kenta Oono and Taiji Suzuki. 2019. Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947* (2019).

[36] Ladislav Rampášek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems* 35 (2022), 14501–14515.

[37] Ladislav Rampášek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a General, Powerful, Scalable Graph Transformer. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 14501–14515. https://proceedings.neurips.cc/paper_files/paper/2022/file/5d4834a159f1547b267a05a4e2b7cf5e-Paper-Conference.pdf

[38] Danial Saber and Amirali Salehi-Abari. 2025. Over-Squashing in GNNs and Causal Inference of Rewiring Strategies. *arXiv preprint arXiv:2508.09265* (2025).

[39] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. 2020. Learning to simulate complex physics with graph networks. In *International conference on machine learning*. PMLR, 8459–8468.

[40] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.

[41] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer, 593–607.

[42] Zhiqi Shao, Dai Shi, Andi Han, Yi Guo, Qibin Zhao, and Junbin Gao. 2023. Unifying over-smoothing and over-squashing in graph neural networks: A physics informed approach and beyond. *arXiv preprint arXiv:2309.02769* (2023).

[43] Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J Sutherland, and Ali Kemal Sinop. 2023. Exphormer: Sparse transformers for graphs. In *International Conference on Machine Learning*. PMLR, 31613–31632.

[44] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. 2021. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522* (2021).

[45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[46] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).

[47] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in neural information processing systems* 34 (2021), 28877–28888.

[48] Jingcheng Yu, Mingliang Zeng, and Qiwei Ye. 2026. FloydNet: A Learning Paradigm for Global Relational Reasoning. *arXiv preprint arXiv:2601.19094* (2026).

[49] Lingxiao Zhao and Leman Akoglu. 2019. Pairnorm: Tackling oversmoothing in gnns. *arXiv preprint arXiv:1909.12223* (2019).

A Algorithms and Implementation Details

This appendix first specifies the two procedures used by Section 4.6. These procedures are the differentiable SSL update and the offline Sandpile-guided rewiring step. The main text gives the equations. The pseudocode below records the order of computation, the fixed preprocessing boundary, and the budget matching used by the experiments.

A.1 SSL and Rewiring Procedures

Algorithm A.1 Sandpile Stabilization Layer.

Inputs: Node embeddings h_v, graph G, capacity τ_v, dissipation η, steps K_{SSL}.
Output: Stabilized embeddings $x^{(K_{\mathrm{SSL}})}$ and congestion proxy $\widehat{u}$.
Init: Set $x_v^{(0)} \leftarrow h_v$ and $\widehat{u}(v) \leftarrow 0$ for all v.
for $k = 0, \dots, K_{\mathrm{SSL}} - 1$ **do**
$o_v^{(k)} \leftarrow [\|x_v^{(k)}\|_2 - \tau_v]_+$ for all v.
$e_v^{(k)} \leftarrow \sigma(\alpha_{\mathrm{gate}} o_v^{(k)} + b_{\mathrm{gate}})$ with $\alpha_{\mathrm{gate}} \geq 0$.
$r_v^{(k)} \leftarrow e_v^{(k)} \frac{o_v^{(k)}}{\|x_v^{(k)}\|_2 + \varepsilon_{\mathrm{num}}} x_v^{(k)}$.
If $d_v > 0$, redistribute $(1-\eta) r_v^{(k)} / d_v$ from v to each non-sink neighbor; otherwise dissipate the release.
Update $x^{(k+1)}$ by subtracting released overflow and adding received overflow.
$\widehat{u}(v) \leftarrow \widehat{u}(v) + e_v^{(k)} o_v^{(k)}$.
end for
Return $x^{(K_{\mathrm{SSL}})}$ and the congestion proxy $\widehat{u}$.

Algorithm A.2 Sandpile-guided rewiring.

Inputs: Graph $G = (V, E)$, congestion scores $\widehat{u}$, edge budget B_{edge}.
Output: A fixed rewired graph for the final training/evaluation loop.
1: Compute original-graph distances exactly by BFS or approximately by landmarks.
2: Select high-congestion source nodes and form feasible non-edge candidates.
3: Score ordered candidate (v, u) by $\mathrm{Score}_G(v, u) = \widehat{u}(v)\, \mathrm{dist}_G(v, u)$.
4: Greedily add the highest-scoring candidates while skipping duplicates and existing edges.
5: **Return** the fixed rewired graph.

A.2 Numerical Details and Edge Construction

For differentiability, the overload uses a smooth positive-part surrogate such as $\phi_\beta(t) = \beta^{-1} \log(1 + \exp(\beta t))$. This surrogate is implemented with stable primitives and satisfies the uniform bound in Appendix B.2. The released vector is normalized with $\|x_v^{(k)}\|_2 + \varepsilon_{\mathrm{num}}$ or an equivalent clipped denominator to avoid division by zero and unstable gradients near the origin.

Capacities are constrained by $\tau_v \geq 0$, implemented as $\tau_v = \mathrm{softplus}(\widetilde{\tau}_v)$ or another positive map. We initialize τ_v from a typical feature norm at the SSL insertion point and optimize it with the model. The gate parameters are shared within a layer unless otherwise stated; α_{gate} is parameterized by a positive map to enforce a monotone overload response. The dissipation parameter η plays the role of the sandpile sink. SSL redistributes only a $(1 - \eta)$ fraction of released overflow and treats the rest as magnitude lost to clipping, saturation, or other capacity-limited effects.

Candidate non-edges are built from high-$\widehat{u}$ nodes to non-neighboring nodes that are far in the original graph. Ties are resolved deterministically by larger distance, then larger congestion score, then lexicographic node order. If the candidate pool is exhausted, the procedure stops early and records the realized budget. For topology-modifying baselines, the final number of added or kept nonzero edges is matched to $B_{\mathrm{edge}} = \rho_{\mathrm{edge}}|E|$ whenever the baseline exposes a density, iteration count, or sparsification threshold. One SSL call costs $O(K_{\mathrm{SSL}}(|V| + |E|)d)$ for hidden dimension d; shortest-path computation belongs to the offline rewiring phase and is not repeated during gradient updates.

B Scalar Relaxation and Scope

This section gives the scalar theory for continuous overload dynamics. It is stated for scalar nonnegative loads, where mass conservation and dissipation have direct meanings. The role of the section is to justify accumulated released overload as a congestion proxy for the implemented layer.

B.1 Scope of the Continuous Relaxation

The analysis separates three levels. The integer Abelian sandpile gives termination, order independence, exact odometers, and exact cut bounds. The scalar dissipative relaxation gives mass decrease, finite-K_{SSL} tail control, and a real-valued accumulated-release proxy. The implemented SSL uses vector norms, learned gates, smooth thresholds, and end-to-end non-convex training.

The practical interpretation follows from this separation. The discrete theory supplies the structural factors of capacity, boundary size, and odometer-like congestion. The vector-valued layer implements differentiable counterparts of these factors through feature-norm capacity, local overflow release, and cut-aware redistribution. The odometer proxy, rewiring score, and collision diagnostics therefore test whether the same bottleneck signal remains useful after relaxation.

The sink has the same accounting role in both the theory and the implementation. In the graph model, it represents loss to the outside of the data region. In SSL, the dissipation parameter η controls the fraction of released overload that is not redistributed to non-sink neighbors. This interpretation covers magnitude loss caused by clipping, saturation, normalization, or capacity-limited transformations. The downstream prediction is made from $x^{(K_{\mathrm{SSL}})}$, while $\widehat{u}$ records cumulative overload activity and is used as a congestion proxy.

B.2 Soft Threshold Approximation

For $\phi_\beta(t) = \beta^{-1} \log(1 + \exp(\beta t))$,

$$0 \leq \phi_\beta(t) - [t]_+ \leq \frac{\log 2}{\beta} \qquad \forall t \in \mathbb{R}.$$

The inequality follows by considering $t \geq 0$ and $t < 0$ separately and using $\log(1 + a) \leq a$ or $\log(1 + a) \leq \log 2$ after factoring out the dominant exponential. Consequently, if release gates are held fixed while comparing the hard and smooth thresholds, replacing the hard positive part by ϕ_β changes each of the node accumulated overload proxy by at most $K_{\mathrm{SSL}} \log(2)/\beta$, and the ℓ_1 gap is at most

$|V^\circ|K_{\mathrm{SSL}}\log(2)/\beta$. If the gates are recomputed from the smoothed overloads, the same conclusion requires the additional Lipschitz factor of $f(o)=\sigma(\alpha_{\mathrm{gate}}o+b_{\mathrm{gate}})o$ on the overload range encountered during training.

B.3 Dissipative Scalar SSL and Finite-K_{SSL} Tail

Consider scalar loads $x^{(k)}\in\mathbb{R}^{V^\circ}_{\geq 0}$, capacities τ, releases $r_v^{(k)}\in[0,[x_v^{(k)}-\tau_v]_+]$, and update

$$x^{(k+1)}=x^{(k)}-r^{(k)}+(1-\eta)\mathbf{P}^\top r^{(k)},$$

where $\mathbf{P}$ is the non-sink random-walk redistribution matrix. Summing over nodes gives

$$M^{(k+1)}=M^{(k)}-\eta\sum_v r_v^{(k)},\qquad M^{(k)}:=\sum_v x_v^{(k)}.$$

Thus, for $\eta>0$, $M^{(k)}$ is a Lyapunov function and $\sum_{k<K_{\mathrm{SSL}}}\sum_v r_v^{(k)}\leq M^{(0)}/\eta$. If $u_{\mathrm{cont}}^{(K_{\mathrm{SSL}})}(v)=\sum_{k<K_{\mathrm{SSL}}}r_v^{(k)}$ and u_{cont} denotes the infinite-step scalar odometer, then

$$\|u_{\mathrm{cont}}-u_{\mathrm{cont}}^{(K_{\mathrm{SSL}})}\|_1=\sum_{k\geq K_{\mathrm{SSL}}}\sum_v r_v^{(k)}=\frac{M^{(K_{\mathrm{SSL}})}-M^{(\infty)}}{\eta}\leq\frac{M^{(K_{\mathrm{SSL}})}}{\eta}.$$

This tail identity justifies logging a finite-K_{SSL} accumulated-release proxy while also explaining why task accuracy need not be monotone in K_{SSL}. Intermediate stabilized feature states can be more or less useful for prediction even though scalar released mass is monotone.

B.4 Relation to Discrete Odometers

In the scalar full-release setting, divisible stabilization is a relaxation of integer chip-firing. The discrete least-action principle minimizes over integer firing vectors $q\in\mathbb{Z}^{V^\circ}_{\geq 0}$ with $z-\mathbf{L}q$ stable, whereas the divisible relaxation allows $q\in\mathbb{R}^{V^\circ}_{\geq 0}$ under the same capacity constraints. Since the relaxed feasible set contains the integer one, the relaxed odometer cannot require more effort than any feasible integer odometer.

$$u_{\mathrm{cont}}(v)\leq u_z(v),\qquad v\in V^\circ.$$

This is why accumulated scalar release is a conservative congestion signal. The implemented SSL adds vector norms, learned gates, and non-convex training, so the paper uses this scalar result only as a bridge for the proxy, not as a transfer of exact sandpile invariants.

C Proof Sketches for Core Claims

This section collects the proof ideas behind the results used in the main text. The goal is auditability under the page limit rather than full textbook proofs.

C.1 Termination, Abelian Property, and Least Action

The reduced Laplacian $\mathbf{L}$ is a nonsingular M-matrix for a connected graph with a sink, so $\mathbf{L}^{-1}\mathbf{1}>0$. Define $\Phi(z)=\mathbf{1}^\top\mathbf{L}^{-1}z$. A legal toppling at v maps z to $z-\mathbf{L}\mathbf{e}_v$ and decreases Φ by one. Since legal topplings keep configurations nonnegative and Φ cannot decrease indefinitely, every legal sequence terminates. Toppling operators commute algebraically, $T_vT_wz=z-\mathbf{L}\mathbf{e}_v-\mathbf{L}\mathbf{e}_w=T_wT_vz$; with termination this gives order-independent stabilization and an order-independent odometer.

For least action, let $q\in\mathbb{Z}^{V^\circ}_{\geq 0}$ be any nonnegative firing vector such that $z-\mathbf{L}q$ is stable. No legal stabilization sequence can fire any vertex more often than q before reaching a stable state; otherwise the first excessive firing would contradict the stability of $z-\mathbf{L}q$. Hence $q\geq u_z$ componentwise, and the ℓ_1 and ℓ_∞ congestion minimality statements follow immediately.

C.2 Green Representation and Critical Group

Writing $\mathbf{L}=\mathbf{D}(I-\mathbf{P})$ with the killed random-walk kernel $\mathbf{P}=\mathbf{D}^{-1}\mathbf{A}$, absorption at the sink implies $\rho(\mathbf{P})<1$. Therefore

$$\mathbf{L}^{-1}=(I-\mathbf{P})^{-1}\mathbf{D}^{-1}=\sum_{k\geq 0}\mathbf{P}^k\mathbf{D}^{-1}.$$

If $z^\circ=\mathrm{stab}(z)$, then $z-z^\circ=\mathbf{L}u_z$, giving $u_z=\mathbf{L}^{-1}(z-z^\circ)$. Since any stable z° satisfies $0\leq z^\circ\leq\deg-\mathbf{1}$, the Green-function upper and lower bounds in Section 3.3 follow.

The quotient $K(G,s)=\mathbb{Z}^{V^\circ}/\mathbf{L}\mathbb{Z}^{V^\circ}$ is finite because $\mathbf{L}$ has full rank and the quotient size is $|\det\mathbf{L}|$. A toppling subtracts a column of $\mathbf{L}$, so stabilization preserves the quotient class. Since the set of stable configurations is finite while $\mathbb{Z}^{V^\circ}_{\geq 0}$ is infinite, stabilization is many-to-one. The critical group is therefore used as a theoretical diagnostic of stabilization invariants and post-stabilization indistinguishability, not as an algorithmic component of SSL or rewiring.

C.3 Separator Rank and Local-Only Corollary

An L-layer local MP-GNN induces a layered computation graph. If every graph path from U to W crosses a separator $\mathcal{B}$, then every computational path from input features on U to output features on W must pass through one of the hidden states on $\mathcal{B}$ at some layer. The total dimension of these separator states is at most $d|\mathcal{B}|(L+1)$, so the Jacobian block factors through a space of that dimension and has rank at most $d|\mathcal{B}|(L+1)$.

The same factorization is unaffected by residual connections, pointwise nonlinearities, feature normalization, or attention weights restricted to existing one-hop neighborhoods. They change the functions computed inside the layered graph, but they do not create a computational path that bypasses $\mathcal{B}$. Such modifications can help optimization or local expressivity while leaving the separator dimension in the rank bound unchanged.

C.4 Cut-Forced Congestion and Rewiring

For $U\subseteq V^\circ$, stability allows at most $C(U)=\sum_{v\in U}(\deg(v)-1)$ chips to remain in U. Therefore at least $[z(U)-C(U)]_+$ chips must leave U during stabilization. Each toppling at $v\in U$ sends exactly $\deg^U_{\mathrm{out}}(v)$ chips across the boundary, so

$$\sum_{v\in U}u_z(v)\deg^U_{\mathrm{out}}(v)\geq[z(U)-C(U)]_+.$$

Using $\sum_{v\in U}\deg^U_{\mathrm{out}}(v)=|\partial U|$ yields the max-odometer cut lower bound. If b_{cut} new simple edges cross from U to $V^\circ\setminus U$, then $|\partial_{G'}U|=|\partial_G U|+b_{\mathrm{cut}}$ and, because the degree of the endpoint in U increases for each added edge, $C_{G'}(U)=C_G(U)+b_{\mathrm{cut}}$. Hence the corresponding forced lower bound is $[z(U)-C_G(U)-b_{\mathrm{cut}}]_+/(|\partial_G U|+b_{\mathrm{cut}})$, which explains why matched-budget rewiring controls are necessary.